Efficient Claude Code: the reference guide

This is the companion to the one-hour session. The deck teaches the habits; this guide explains them, covers the topics the deck only mentions, and gives you templates to copy. If you only read one thing, read the cheat sheet. If you want to know why it works, read this.

Commands and settings change between Claude Code releases. Everything here was checked on version 2.1.287. If something doesn't match, run `/help` on your version.

1. The one idea: the meter

Claude doesn't remember anything between messages. Every time you send a message, Claude Code sends the whole conversation again: the system prompt, tool definitions, skills, MCP server tools, CLAUDE.md, every file Claude read, every command output, and every message so far. Think of it as a taxi meter that never resets until you start a new session.

Three consequences follow, and every rule in this guide comes from them.

Cost depends on context size, not message length. In our Run A recording, a one-line "I'll go test" re-read 93k tokens. The closing "thanks!" re-read 130k.

Most of the bill is re-reading, not writing. In Run A, about 52% of the cost was re-reading the cached conversation, 22% was caching new context, and only 26% was Claude's actual output. Prompt caching makes re-reading cheaper, but not free.

A fresh session isn't empty. Before you type anything, a session is already around 50k tokens: system prompt, tools, skills, MCP servers and CLAUDE.md. All of that rides along on every message.

You can see your own meter at any time with `/context`. Try it: open a fresh session and run `/context`, ask Claude to read a large file, run `/context` again, then send "ok". The total doesn't go down. Every later message pays for that file.

2. The ten Golden Rules

These are designed to be followed blindly. You don't need to understand the meter to benefit from them, but each one explains itself once you do.

Rule 1: One task, one session

When the task changes, run `/clear`. Don't use one session all day for a feature, a git question, yesterday's bug and a thank-you. A quick unrelated question asked inside a 140k session costs as much as a real task, and the answer then sits in your feature's context too. Ask side questions in a separate chat, or after clearing.

Signs it's time to clear: you've finished what you set out to do, you're about to start something unrelated, or `/context` shows the session has grown past roughly 100k and the work ahead doesn't need what's in it.

Rule 2: Plan before you build

For anything that touches more than one file, write a short spec and use plan mode before any code is written.

1. Write a spec of 10 to 20 lines (template in section 4). Include what's out of scope and when it's done.
2. Turn on plan mode with `Shift+Tab` (press until the mode indicator says plan). Claude researches and proposes, but doesn't edit.
3. Read the plan. Make at most one targeted correction.
4. Approve it.

This is the biggest single win we measured. Run A pasted a ticket and said "implement this", then needed four rounds of scope corrections. Run B wrote a spec and planned first, and needed none. The spec took a person about 15 minutes. That's real time; it's still cheaper than four correction rounds on a growing context.

What you leave out of the spec is what you'll correct later. Run B's only follow-up prompts were styling preferences the spec didn't mention.

Rule 3: Pick the right model

- **Sonnet** is the default for implementation.
- **Opus** is for planning, architecture and hard debugging.
- `/model opusplan` uses Opus while in plan mode and Sonnet once you approve. This is the easiest way to get both.
- **Haiku** is for trivial, self-contained jobs in a fresh session.

Switch models only right after `/clear`. Prompt caching is per model. If you switch models mid-session, the whole conversation has to be cached again on the new model. In Run B, switching to Haiku for the final commit re-cached a 73k context; that cost three quarters of everything Haiku spent, and saved almost nothing compared with staying on Sonnet.

Rule 4: Point, don't make it search

Searching costs tokens: every file Claude opens to look around stays in context for the rest of the session. If you know where the code is, say so, and name an existing pattern to copy. Use `@path/to/file` to attach files directly.

- Before: "Add AI triage to tickets."
- After: "Follow the pattern in `@lib/tickets/categorize.ts`. Changes go in `@lib/tickets/service.ts` and `@lib/db/schema.ts`."

Naming a pattern also stops duplicates. In Run B, Claude reused the existing AI client wrapper and offline fallback immediately because the prompt pointed at them.

Rule 5: Keep CLAUDE.md short

CLAUDE.md is loaded on every message, so you pay for every line on every turn. Only include what's needed on every task:

- the commands to run tests, typecheck and build;
- hard rules ("LLM calls only through `lib/ai/client.ts`");
- a pointer to your architecture map, rather than the map itself;
- guardrails, such as "Work only inside this repository" and "Ask before installing, removing or upgrading packages."

Leave out onboarding text, API documentation, history and style essays. Link to them instead; Claude will read them when a task needs them.

The guardrails matter. In Run A, Claude went into the home folder to delete a stray file and installed a package without asking. Neither was requested. A short CLAUDE.md template is in `starter-kit/CLAUDE.md`. For hard limits, use permission rules as well (`/permissions`), since CLAUDE.md is guidance, not enforcement.

Rule 6: Two strikes, then rewind

If you've corrected the same thing twice and it's still wrong, stop arguing. Press `Esc` twice or run `/rewind` to go back to before the bad turn, and rewrite that prompt with the detail that was missing.

Arguing is the most expensive thing you can do. Each round adds to the meter, and the failed attempts stay in context and keep steering the next answer. Rewinding removes them.

Rule 7: Facts, not feelings

Frustration feels like information, but the model can't act on it. Describe the behaviour you see and the behaviour you want, in one sentence.

- Before (a real message from Run A): "You should be smart enough to realize this. No one is going to find the priority by clicking the details for each ticket."
- After: "Show the priority badge on each ticket card in the list. Leads must be able to change the priority on the detail page."

For bugs, use the pattern *I did X, expected Y, got Z*: "After clicking Regenerate, the summary only updates when I reload the page."

Rule 8: Trim what you paste

Anything you paste stays in context for the rest of the session, so it's re-sent with every message after it. Paste the failing test name and the 10 to 20 error lines, not 2,000 lines of

build output. Add this to CLAUDE.md so Claude's own command output stays small too: "When you run tests or builds, report only failures."

You can also run a command yourself with `!` (for example `! npm test`) when you want to see the full output without discussing it.

Rule 9: Hand off, don't drag on

At a milestone (backend done, research finished, a bug found), write a short handoff note, clear, and continue from the note.

1. Run `/handoff`. The skill in `starter-kit/.claude/skills/handoff/` writes at most 25 lines to `.claude/handoff.md`: goal, what's done, decisions, next steps, known issues.
2. Run `/clear`.
3. Start the next part with "Read `@.claude/handoff.md` and continue with the next step."

If you really can't clear, use `/compact` with instructions about what to keep, for example `/compact keep the plan, the files changed and the failing tests; drop the exploration`. Plain `/compact` keeps whatever it judges important, which may not be what you need.

Plan mode can also clear for you. With `"showClearContextOnPlanAccept": true` in `~/.claude/settings.json`, each approve option offers to clear the planning context first and start a fresh session with only the plan. That's what Run B did between planning and building.

Rule 10: No unrequested docs

AI-generated summary files pile up fast: `IMPLEMENTATION_SUMMARY.md`, `CHANGES.md`, `NOTES_v2.md`. Nobody reads them, and later sessions read them back in as context. Add one line to CLAUDE.md: "Do not create markdown files unless asked." See section 9 for what to keep.

The same idea applies to MCP servers: each connected server's tool descriptions sit in context on every message. Turn off the ones the current task doesn't need (`/mcp`).

3. The workflow: requirement to pull request

This is the Run B flow, written as a recipe. Use it for any task that touches more than a couple of files.

1. **Write the spec** in `docs/specs/<ticket>.md` using the template in section 4.
2. **Start fresh** with `/clear`, or a new session. Run `/model opusplan`.
3. **Plan.** Turn on plan mode and prompt: "Read `@docs/specs/<ticket>.md` and plan the implementation. Existing patterns to follow: `@…`. Work only inside this repo. List files and steps only, no code."
4. **Review the plan.** At most one targeted correction, then approve. If you've enabled the clear-context option, choose it.
5. **Build.** Claude implements on Sonnet. Ask it to run tests, typecheck and build, and report only failures.

6. **Check it yourself** in the browser or with a quick manual test. Collect every issue first, then send them in one message, each as a fact.
7. **Hand off** if the work isn't finished and the context is large: `/handoff`, `/clear`, continue from the note.
8. **Ship.** "Commit with a conventional commit message, push the branch and open a PR with gh. Use the spec for the description."

For a one-file fix, skip the spec and plan: point at the file, describe the change precisely, and clear when it's done.

4. Writing specs and prompts

A spec is a prompt that's been thought through. It doesn't need to be long; Run B's was 15 lines. The template is in `starter-kit/docs/specs/_template.md`:

```
# Spec: <name> (<ticket>)

Goal: <one or two sentences: what changes for the user>

- Data: <schema changes, migrations>
- Logic: <where it goes, which existing pattern to follow>
- API: <endpoints, inputs, errors>
- UI: <what appears where, empty and error states>
- Tests: <the cases that must be covered>
- Out of scope: <what not to do, even if it seems helpful>
- Done when: <the checks that must pass>
```

For any prompt, spec or not, check these five things:

1. **Purpose.** What problem does this solve, and for whom?
2. **Context.** Which files, which patterns, which constraints?
3. **Specifics.** Behaviour, edge cases, and anything Claude would otherwise guess.
4. **Out of scope.** The anti-goals. Most people never write these, and they prevent the most waste.
5. **Done when.** How Claude, and you, will know it's finished.

The session's "fix a prompt" exercise shows the difference. "Add search to the tickets page" leaves Claude to guess the fields, the matching rules, the location, how it interacts with filters, and what's out of scope. The rewritten version names all of that in four sentences.

5. Context management, command by command

Command	What it does	When to use it
`/context`	Shows what's in the context and how big it is	Before deciding whether to continue or clear
`/clear`	Starts a fresh conversation in the same terminal	Every time the task changes
`/compact <instructions>`	Summarises the conversation to shrink it	When you can't clear; always say what to keep
`/rewind` or `Esc` `Esc`	Goes back to before an earlier turn	After two failed corrections
`/handoff`	Writes a short note to continue from (starter kit skill)	At a milestone, before clearing
`/model <name>`	Switches model: `sonnet`, `opus`, `haiku`, `opusplan`	Right after `/clear`
`Shift+Tab`	Cycles permission modes, including plan mode	Before any multi-file change
`/cost`, `/usage`	Session cost and token use; plan limits	Before clearing a big session, and daily
`/mcp`	Manages MCP servers	Turn off servers the task doesn't need
`@path`	Attaches a file to your prompt	Whenever you know which file matters
`! <command>`	Runs a shell command yourself	When you want output without a discussion

Remember that `/clear` starts a new session, so `/cost` resets with it. If you're tracking usage, run `/cost` before you clear.

6. Large codebases: an architecture map, not a graph

It's tempting to generate a full graph of a large codebase so Claude "understands" it. In practice, a full graph goes stale, someone has to maintain it, and it gets loaded into context when it isn't needed.

A one-page architecture map gives most of the benefit. It should list each top-level folder with one line on what lives there, the main request flows (for example "create ticket: route → validation → service → AI enrichment"), and the rules that apply everywhere. There's a 20-line example in `starter-kit/docs/architecture.md`, from the demo app used in the session.

Reference it from CLAUDE.md ("Architecture map: `docs/architecture.md`. Read it before exploring the codebase.") rather than pasting it in. Claude reads it once per task when needed instead of on every message.

Beyond the map, the cheapest way to handle a large codebase is still Rule 4: tell Claude which files matter. Two `@` mentions beat twenty searches.

7. Subagents and multi-agent setups

A subagent is a separate Claude instance that does a job in its own context and reports back a summary. Claude Code can start them itself (for example to explore a codebase), and you can define your own in `.claude/agents/` (see `/agents`).

What they're good for: keeping noisy work out of your main context. A subagent can read 30 files to answer "where is rate limiting handled?" and hand back three lines. Your main session never pays to re-read those 30 files.

What they cost: the subagent's own work is still paid for. Running several agents in parallel multiplies total usage; it's faster, not cheaper.

The rule of thumb for a team short on limits: use subagents for exploration you'd otherwise do in the main session. Avoid parallel multi-agent setups for routine work; save them for genuinely parallel, independent tasks where speed matters more than usage.

8. Skills and custom commands

A skill is a folder with a `SKILL.md` file that tells Claude how to do a specific job. Project skills live in `.claude/skills/<name>/`, personal ones in `~/.claude/skills/<name>/`. Only each skill's short description sits in context; the full instructions load when the skill is used. You can invoke one by name, like `/handoff`, or Claude can pick it when the description matches the task.

Turn anything you type more than twice a week into a skill. Good candidates:

- `/handoff`: the milestone note from Rule 9 (in the starter kit);
- `/spec`: draft a spec from a ticket using your template, for you to edit;
- `/ship`: run checks, commit, push and open a PR in your team's format.

Keep skills short and specific. A skill is also a prompt, and the same rules apply: purpose, context, specifics, out of scope, done when.

9. Keeping AI-generated markdown under control

Keep three kinds of markdown, and delete the rest:

- **Specs** in `docs/specs/`. They're the record of what was asked and why, and they make good PR descriptions.
- **The architecture map**, in `docs/architecture.md`.
- **Handoff notes** in `.claude/handoff.md`, overwritten each time and listed in `.gitignore`.

Everything else (change summaries, implementation notes, "analysis" files) should either go in the PR description or not exist. Add "Do not create markdown files unless asked. No

change summaries in files." to CLAUDE.md, and delete any that already exist. Old notes are worse than none: Claude may read them back in and trust them over the current code.

10. Measuring your own usage

Without a number, "it feels like we use less" is just a feeling. Two simple ways to measure:

- **Daily:** note your session and weekly usage percentage from `/usage` at the start and end of each working day, this week and next.
- **Per task:** run `/cost` before you clear a big session and write down the total. Compare similar tasks before and after.

Claude Code also writes every session to a log under `~/.claude/projects/<project-folder>/`, one JSONL file per session, with token usage for every message. That's how we measured the A/B comparison, and it works even when you forget to run `/cost` before clearing.

11. Our A/B results, with the caveats

We implemented the same ticket (TD-142: AI summary, suggested and editable priority, list sorting and filtering) twice from the same starting commit, on the same machine and Claude Code version. Both runs ended with the same features, passing tests and an open PR.

	Run A: how we usually work	Run B: Golden Rules
Cost, priced from the session logs	$2.67	$1.15
Wall time	46 min	27 min
Prompts typed	9	6
Scope corrections	4 rounds	0
Peak context	130k	89k
Code changed	32 files, +1,012	27 files, +797

57% lower cost for the same feature set. Run A's own `/cost` reported $2.92; Run B's build session reported $0.91, plus about $0.28 for the planning session, which was cleared before `/cost` was run. Both methods give a cut of over 55%.

The honest costs on Run B's side:

- Writing the spec took a person about 15 minutes.
- Run B's three follow-up prompts were styling preferences that weren't in the spec.
- Run B's first session committed, pushed and opened the PR before those tweaks, and Claude checked the API rather than clicking through the UI.
- Run A's detour into the home folder was triggered by a stray file that it then deleted, so Run B never faced the same temptation.

- Session and weekly percentages aren't comparable between the runs, because Run A's starting percentage wasn't recorded. Use the cost figures.

`deck/replay.html` steps through both runs prompt by prompt, built from the two runs' session logs.